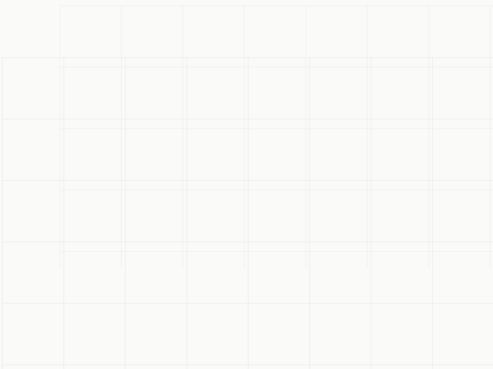


Summer Research Bootcamp

Multi-Agent Systems Research Program



Communication Protocols for Multi-Agent LLM Systems:

An Empirical Study

Principal Investigator	Xihao Xie
Project Assistant	Chang Liu
Research Area	Agentic AI, Multi-Agent Systems, Large Language Models
Duration	3 Weeks

1 Introduction

Large Language Models (LLMs) have recently evolved from standalone conversational agents into complex systems composed of multiple collaborating agents. Systems such as AutoGen, CAMEL, MetaGPT, CrewAI, and LangGraph demonstrate that teams of specialized agents can often accomplish tasks that are difficult for a single model.

Despite rapid adoption, there is little systematic understanding of how AI agents should communicate. Most existing systems focus on:

- Adding more agents
- Adding more tools
- Building larger workflows

However, the communication protocol itself may be one of the most important design choices. This project investigates the following question:

How should AI agents communicate in order to maximize performance, reliability, and cost-effectiveness on complex knowledge-work tasks?

Students will form teams to design, implement, and evaluate multiple communication protocols while keeping the model, benchmark tasks, tool access, evaluation criteria, and execution budget as consistent as possible.

The project culminates in:

- An open-source benchmark for multi-agent communication
- A multi-agent experimentation platform
- A research-style empirical report
- Academic-style presentations and demos

2 Motivation

Human organizations have spent thousands of years discovering effective communication structures:

- Military hierarchies
- Corporate management
- Scientific collaborations
- Democratic decision-making
- Market-based coordination

Modern multi-agent AI systems face many of the same coordination challenges:

- Information sharing
- Task decomposition
- Conflict resolution
- Error correction
- Resource allocation

Yet the field lacks rigorous empirical evidence regarding which communication structures are most effective for AI agents. This project addresses that gap by comparing multiple communication protocols under controlled experimental conditions.

3 Research Questions

– Primary Research Question

How do communication protocols influence the effectiveness of multi-agent LLM systems?

– Main Research Questions

- RQ1: Which communication protocol produces the highest-quality outputs?
- RQ2: Which communication protocol achieves the best cost-quality tradeoff?
- RQ3: How does communication overhead affect output quality, latency, and cost?
- RQ4: What failure modes emerge under different communication structures?

– Optional Extended Research Questions

- Optional RQ5: Which task categories benefit most from multi-agent collaboration?
- Optional RQ6: How does the number of agents affect performance under a fixed communication protocol?

4 Hypotheses

- H1: Communication protocols with explicit coordination mechanisms will outperform loosely coordinated protocols in output quality and reliability.
- H2: Shared-blackboard protocols improve output completeness by preserving intermediate information across agents, but they may also increase noise accumulation and communication overhead.
- H3: Debate protocols reduce hallucinations and reasoning errors by forcing agents to critique competing answers, but they increase token cost and runtime.
- H4: Hierarchical manager-worker protocols scale better to complex tasks by centralizing task decomposition and coordination, but the manager may become an information bottleneck.
- Optional H5: Communication costs eventually outweigh collaboration benefits when additional messages no longer improve output quality.
- Optional H6: Specialized role-based agents outperform general-purpose agents when the communication protocol is fixed.

Note: Optional H6 requires an additional ablation in which multiple general-purpose agents collaborate under the same communication protocol. It should not be treated as a main hypothesis unless that ablation is implemented.

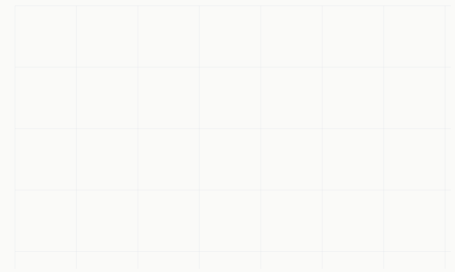
5 Background Reading

– Foundational Agent Papers

- ReAct: Reasoning and Acting in Language Models
- Reflexion: Language Agents with Verbal Reinforcement Learning
- Toolformer: Language Models Can Teach Themselves to Use Tools
- Voyager: An Open-Ended Embodied Agent

— Multi-Agent Papers

- AutoGen: Enabling Next-Generation Multi-Agent Systems
- CAMEL: Communicative Agents for Mind Exploration
- MetaGPT: Meta Programming for Multi-Agent Collaboration
- ChatDev: Communicative Agents for Software Development
- AgentVerse: Multi-Agent Simulation Framework



— Optional Advanced Reading

- Organizational Theory
- Distributed Systems
- Collective Intelligence
- Swarm Intelligence
- Mechanism Design

6 Project Architecture

The goal is to compare communication structures rather than models. All experiments should keep the following factors fixed whenever possible:

- Same LLM and model version
- Same benchmark tasks
- Same tool access
- Same evaluation criteria and scoring rubric
- Same temperature and sampling settings
- Same maximum token budget and maximum interaction budget
- Same logging format and result schema

The main independent variable is the communication protocol. Role-specific prompts may vary across protocols, but they must follow standardized templates and be documented for reproducibility.

— Core system pipeline:

- Input task and rubric
- Protocol controller
- Agent execution and communication
- Tool use, if required by the task
- Final answer generation
- Automatic logging
- Human or rubric-based evaluation

7 Agent Roles

Students implement five reusable agent types. These roles should be reused across protocols to reduce confounding variables.

Agent Role	Responsibilities	Outputs
Planner	Task decomposition; goal generation; workflow planning	Subtasks; execution plans
Researcher	Information gathering; source collection; fact retrieval	Evidence; citations; supporting material
Analyst	Synthesis; interpretation; reasoning	Conclusions; comparisons
Critic	Verification; error detection; challenging assumptions	Corrections; feedback
Writer	Final report generation; organization; presentation	Final deliverable artifact

8 Communication Protocols

Each protocol must be implemented with an explicit execution definition. For every protocol, students should specify message flow, visibility, maximum rounds, and termination condition.

– Protocol o: Unstructured Group Chat

A loosely coordinated multi-agent baseline in which all agents can freely discuss the task without a predefined communication order or strict control flow.

Field	Specification
Message flow	All agents receive the task prompt and may respond in any order. A Writer or final judge produces the final answer from the transcript.
Visibility	All agents can see the shared conversation history.
Maximum rounds	Maximum of 3 discussion rounds or an equivalent token budget.
Termination condition	Stop when the maximum number of rounds is reached or when the Writer declares the answer ready.
Advantages	Flexible and easy to implement.
Disadvantages	Difficult to control; may produce redundant discussion, role confusion, or premature consensus.

– Protocol A: Sequential Handoff

Planner -> Researcher -> Analyst -> Writer.

Field	Specification
Message flow	The Planner creates a task plan. The Researcher receives the plan and collects evidence. The Analyst receives the plan and evidence, then produces conclusions. The Writer receives all previous outputs and generates the final answer.
Visibility	Each agent sees the original task and outputs from previous agents only. No backward communication is allowed.
Maximum rounds	One forward pass through the agent chain.
Termination condition	Stop after the Writer produces the final answer.
Advantages	Simple, low cost, and easy to debug.
Disadvantages	Information bottlenecks; early mistakes may propagate downstream.

— Protocol B: Shared Blackboard

All agents write to a shared workspace.

Field	Specification
Message flow	Agents contribute plans, evidence, analyses, critiques, and draft text to a shared blackboard. The Writer synthesizes the final answer from the blackboard.
Visibility	All agents can read the task prompt and the full blackboard state.
Maximum rounds	Maximum of 2-3 contribution rounds.
Termination condition	Stop when all agents have contributed or when the maximum number of rounds is reached.
Advantages	Shared context, transparency, and stronger information retention.
Disadvantages	Noise accumulation, redundant content, and increased communication overhead.

— Protocol C: Manager-Worker

A manager assigns subtasks. Worker agents complete subtasks and report results.

Field	Specification
Message flow	The Manager decomposes the task, assigns subtasks to Planner, Researcher, Analyst, Critic, and Writer roles, collects their outputs, resolves conflicts, and instructs the Writer to produce the final answer.
Visibility	The Manager sees all messages. Workers see the task and their assigned subtask; optionally, they may see relevant summaries from other workers.
Maximum rounds	One planning round, one worker execution round, and one synthesis round. Optional second revision round if the Critic detects major issues.
Termination condition	Stop when the Manager approves the final answer or when the maximum interaction budget is reached.
Advantages	Structured, scalable, and suitable for complex tasks.
Disadvantages	Manager bottleneck and possible loss of details during summarization.

— Protocol D: Debate

Multiple agents independently solve the task and then critique each other.

Field	Specification
Message flow	Agents first produce independent answers. They then critique competing answers and respond to critiques. A Writer or Judge synthesizes the final answer.
Visibility	Agents initially work independently. During debate, they can see other agents' answers and critiques.
Maximum rounds	One independent answer round and 1-2 critique rounds.
Termination condition	Stop after the critique rounds or when the Judge determines that remaining disagreements are resolved.
Advantages	Robustness, error reduction, and better assumption checking.
Disadvantages	Expensive in tokens and runtime; may overemphasize argumentative style.

— Protocol E: Voting

Multiple agents independently produce answers. Majority vote or scoring determines the final output.

Field	Specification
Message flow	Agents independently generate candidate answers. A voting or scoring mechanism selects the best answer or combines top-ranked answers.
Visibility	Agents do not see each other's answers before voting. The Judge or voting module sees all candidate answers.
Maximum rounds	One answer generation round and one voting/scoring round.
Termination condition	Stop after the vote or score aggregation is completed.
Advantages	Simple, parallelizable, and easy to compare.
Disadvantages	May reinforce shared mistakes; weak if all agents have similar failure modes.

— Protocol F: Dynamic Task Allocation

Agents negotiate responsibilities and adapt the workflow during execution.

Field	Specification
Message flow	Agents propose subtasks, negotiate responsibilities, request assistance when needed, and dynamically revise the plan based on intermediate results.
Visibility	Agents can see the shared negotiation history and relevant intermediate outputs.
Maximum rounds	Maximum of 3-4 negotiation and execution rounds.
Termination condition	Stop when agents agree that the task is complete or when the maximum interaction budget is reached.
Advantages	Adaptive and flexible for open-ended tasks.
Disadvantages	Complex implementation; may create excessive communication overhead.

9 Benchmark Design

Students will create 30 benchmark tasks, with 5 tasks in each of the six categories below. Each category should include a mixture of easy, medium, and hard tasks. The benchmark should be challenging enough to require multi-step reasoning, but feasible enough to evaluate within the 3-week schedule.

Category	Example Tasks
Literature Review	Summarize a research field; compare methods; identify research gaps
Technical Analysis	Explain algorithms; compare architectures; analyze design choices
Software Engineering	System design; architecture reviews; code planning
Market Research	Competitive analysis; trend identification; product comparison
Educational Content	Lesson creation; tutorial design; concept explanation
Strategic Planning	Product launch plans; resource allocation; project roadmaps

— Each benchmark task must include the following fields:

- Task ID
- Category
- Difficulty level: Easy, Medium, or Hard
- Prompt
- Required output format
- Ground truth criteria or expected answer criteria
- Required evidence or citation expectations
- Scoring rubric
- Tool requirement: required, optional, or prohibited
- Expected failure risks

— Task Template

Field	Description
Task ID	Unique identifier such as LR-01 or SE-04
Category	One of the six benchmark categories
Difficulty	Easy, Medium, or Hard
Prompt	The exact instruction given to the system
Required Output Format	Bullets, table, report, comparison matrix, or another specified format
Ground Truth / Evaluation Criteria	Key facts, claims, reasoning steps, or deliverable requirements that must be satisfied
Required Evidence	Expected sources, citations, calculations, or supporting material
Scoring Rubric	Criteria used by human evaluators or automated evaluators
Tool Requirement	Whether web search or another tool is required, optional, or prohibited
Expected Failure Risks	Likely failure modes such as hallucination, missing evidence, or poor coordination

10 Experimental Design

The experimental design compares a single-agent baseline, a loosely coordinated multi-agent baseline, and multiple structured communication protocols. The goal is to isolate the effect of communication protocol as much as possible.

— Baseline and Main Conditions

Condition	Protocol	Description
Baseline	Single Agent	A single LLM agent completes the task without multi-agent communication.
Condition 0	Unstructured Group Chat	Multiple agents freely discuss without predefined communication structure.
Condition 1	Sequential Handoff	Planner -> Researcher -> Analyst -> Writer.
Condition 2	Shared Blackboard	All agents write to and read from a shared workspace.

Condition	Protocol	Description
Condition 3	Manager-Worker	A manager coordinates specialized workers.
Condition 4	Debate	Agents independently answer and then critique each other.
Condition 5	Voting	Agents independently answer and a vote or score selects the final output.
Condition 6	Dynamic Allocation	Agents negotiate responsibilities and adapt the workflow during execution.

— Optional Ablations

- Generalist-agent ablation: Replace specialized roles with identical general-purpose agents under the same protocol to test Optional H6.
- Agent-number ablation: Fix one protocol and vary the number of agents to study when additional agents stop improving performance.
- Round-budget ablation: Fix one protocol and vary the maximum number of communication rounds to study cost-quality tradeoffs.

— Recommended Experimental Procedure

1. Run the single-agent baseline on all benchmark tasks.
2. Run each communication protocol on the same tasks using the same LLM, tool access, and evaluation settings.
3. Log all prompts, intermediate messages, tool calls, token counts, runtime, and final outputs.
4. Evaluate outputs using the shared scoring rubric.
5. Analyze quality, cost, communication overhead, and failure modes.

11 Metrics

Metrics should be defined before full experiments begin. The project should report quality, efficiency, communication, and failure-analysis metrics.

Metric	Operational Definition
Accuracy	Correctness of the final output. Can be scored from 1–5 and then normalized to [0, 1], or calculated directly as correct required claims divided by total required claims.
Completeness	Coverage of required information. Calculate as covered required criteria divided by total required criteria.
Helpfulness	Human rating from 1–5 based on whether the answer is useful for the intended user; normalized to [0, 1] before computing the overall quality score.
Hallucination Rate	Unsupported factual claims divided by total factual claims.
Overall Quality Score	Weighted average of accuracy, completeness, helpfulness, and hallucination penalty.
Runtime	Total completion time in seconds per task.
Cost	Input tokens plus output tokens, or estimated API cost per task.
Tool Usage	Number of external tool calls per task.
Message Count	Total number of inter-agent messages per task.

Metric	Operational Definition
Communication Density	Messages per agent per task, or total messages divided by number of active agents.
Agreement Rate	Frequency of consensus among agents or judges.
Critique Acceptance Rate	Accepted critiques divided by total critiques.
Quality-Cost Ratio	Normalized quality score divided by token cost or estimated API cost.

– Recommended quality scoring formula:

Before computing the overall score, all component metrics must be normalized to the range [0, 1]. If Accuracy or Helpfulness is scored on a 1–5 scale, it should be converted using:

$$\text{Normalized Score} = \frac{\text{Raw Score} - 1}{4}$$

$$\text{Overall Quality Score} = 0.35 \times \text{Accuracy}_{\text{norm}} + 0.30 \times \text{Completeness}_{\text{norm}} + 0.20 \times \text{Helpfulness}_{\text{norm}} + 0.15 \times (1 - \text{Hallucination Rate})$$

Completeness and Hallucination Rate should already be computed as proportions in the range [0, 1]. Teams may adjust weights if they justify the change before experiments begin. The same formula and normalization method must be used across all protocols.

12 Failure Analysis

Failure analysis is required, not optional. Each team must manually analyze at least 5 failed or low-scoring cases, or at least 20% of the benchmark tasks, whichever is larger. Failures should be classified using the taxonomy below.

Failure Type	Description
Coordination Failure	Agents work at cross-purposes or duplicate each other unnecessarily.
Communication Failure	Important information is lost, ignored, or not passed to the agent that needs it.
Role Confusion	Agents perform responsibilities outside their assigned roles or fail to perform their assigned roles.
Hallucination Propagation	Incorrect information appears in one agent message and spreads to later outputs.
Premature Consensus	Agents agree too early without adequate verification or critique.
Over-Collaboration	Excessive discussion increases cost without improving quality.
Manager Bottleneck	In hierarchical protocols, the manager omits details, misassigns tasks, or blocks useful collaboration.
Noise Accumulation	Shared workspaces become cluttered with irrelevant, redundant, or misleading information.

Failure-analysis outputs should include the protocol, task category, failure type, short explanation, supporting transcript excerpt, and recommended fix.

13 Schedule

The schedule assumes a 3-week research bootcamp. Students should work in teams, with each team responsible for

one or two communication protocols and a shared benchmark/evaluation pipeline.

Time	Activities
Week 1, Day 1	Project kickoff; introduction to agentic AI and multi-agent systems; tool and framework setup.
Week 1, Day 2	Paper discussion; review of communication protocols; assign teams and protocols.
Week 1, Day 3	Benchmark task design; define task templates and scoring rubrics.
Week 1, Day 4	Single-agent baseline implementation; logging format design.
Week 1, Day 5	Pilot evaluation; metric validation; prompt standardization.
Week 2, Days 6-7	Each team implements assigned protocol(s); shared code interface and configuration files.
Week 2, Day 8	Infrastructure testing; debugging; ensure all protocols produce standardized logs.
Week 2, Day 9	Pilot experiments across a small subset of tasks; revise prompts and execution budgets.
Week 2, Day 10	Freeze benchmark, prompts, metrics, and protocol definitions.
Week 3, Days 11-12	Full experiments across all tasks and protocols.
Week 3, Day 13	Statistical analysis, ablation studies, and cost-quality analysis.
Week 3, Day 14	Failure analysis, figure creation, report writing.
Week 3, Day 15	Final presentations, demos, and research discussion.

14 Reproducibility Requirements

Each team must submit enough material for another student to reproduce the experiments. Required reproducibility artifacts include:

- Source code
- Prompt templates for each agent role and protocol
- Configuration files, including model name, temperature, token budget, tool settings, and maximum rounds
- Benchmark tasks and scoring rubrics
- Raw experiment logs, including intermediate agent messages and tool calls
- Scoring results and evaluator notes
- Analysis notebooks or scripts
- README instructions for reproducing the experiments
- Known limitations and implementation notes

All teams should use a shared logging schema so that results can be aggregated across protocols.

15 Expected Figures

1. Architecture diagrams for each protocol
2. Protocol comparison chart
3. Cost vs. quality tradeoff plot
4. Communication overhead vs. output quality plot
5. Communication network graphs
6. Failure distribution charts

7. Ablation study results
8. Task-category performance breakdown

16 Report Outline

1. Abstract
2. Introduction
3. Related Work
4. Methodology
5. Benchmark Design
6. Experimental Setup
7. Results
8. Failure Analysis
9. Discussion
10. Limitations
11. Future Work
12. Conclusion

17 Expected Contributions

The project may contribute:

1. A benchmark for multi-agent communication.
2. A reusable evaluation framework.
3. An empirical comparison of communication protocols.
4. A failure taxonomy for multi-agent systems.
5. Practical recommendations for future agent architectures.
6. Reproducible experiment logs and prompt templates for future research.

18 Success Criteria

Level	Criteria
Minimum Success	Functional multi-agent framework; benchmark dataset; experimental results for the single-agent baseline and at least three communication protocols.
Strong Success	All main protocols implemented; clear empirical findings; open-source release; complete failure analysis; workshop-style report.
Exceptional Success	Publication-quality empirical study; publicly released benchmark with clear documentation; reusable framework designed for future research projects.